real-tr-p8-rdallreduce

An AgentMPI run analysis

Abstract

This run is the allreduce-algorithm rung of the parallel book-translation ablation: eight real Cursor subagents translate *A Study in Scarlet* at $p = 8$, identical to `real-tr-p8-full` in every respect except that the glossary agreement step uses `recursive_doubling` instead of the default `reduce_bcast`. The operator was the exact, idempotent `UNION`, so the divergence hazard that makes recursive doubling interesting was not armed: all eight `coll.allreduce` events record `divergence_risk: false`, and all eight ranks left the collective holding the identical 206-term glossary, digest `c8b0a1300f57`. What the run measures instead is the price of the algorithm, and the measurement is sharper than the closed form suggests. Halving the round count from six to three bought **59 ms**: once every rank had arrived, the collective cost 0.430 s against the baseline's 0.489 s. Meanwhile 96% of its 11.43 s wall time went on waiting for the last rank's terminology agent, and it paid for those three rounds with 24 messages instead of 14. The single most important thing to take away is a fact the cost model states and the naive reading of "latency-optimal" obscures: **recursive doubling does not reduce the fold depth**. Both algorithms fold to depth $\lceil \log_2 p \rceil = 3$: three at every rank here, and three at the root of the baseline's reduce. The extra three rounds `reduce_bcast` spends go on a broadcast that performs no operator applications at all. Recursive doubling therefore offers a lossy operator no fidelity improvement whatsoever, while raising the total number of operator applications from $p - 1 = 7$ to $p \log_2 p = 24$ and admitting the possibility that the population disagrees. For an exact operator, as here, the choice is nearly free in both directions; for a semantic one it is a bad trade, and that corner of the sweep is the one nobody ran.

1 What this run is

property	value
run	`real-tr-p8-rdallreduce`
experiment	`translation`
campaign label	`real-tr`
declared world size	8
ranks appearing in the log	8
executors	`broker` × 8, `worker` × 31
events	349
wall time	158.08 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	6.21×	total busy time over wall time
parallel efficiency	77.6%	achieved parallelism per rank
idle fraction	22.4%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	14.8%	time with exactly one rank busy
load imbalance	1.09×	slowest rank's busy time over the mean
coordination share	16.1%	rank-seconds blocked in collectives, of those available
coordination in flight	23.3%	wall clock with any rank inside a collective
rank reattachments	31	fresh agent processes that took over a rank role
agent calls	24	invocations that reached a model
agent latency p50 / p95	48.52 / 59.52 s	per invocation
messages	66	58 eager, 8 rendezvous
tokens sent	31,373	8,912 deferred under rendezvous
tokens in / out	43,340 / 9,118	prompt and completion
cost	$0.2668	0.0293 $/kTok out

3 What the analysis notices

- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 14: the collective's own accounting disagrees with the traffic it produced.
- [ok] 31 rank reattachment(s) occurred, up to incarnation 5: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 7 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	121.79	77.0	3	3	10	10	9,114	0.0	finalized
1	123.34	78.1	3	3	7	7	2,169	0.0	finalized
2	112.95	71.5	3	3	9	9	3,456	0.0	finalized
3	113.82	72.0	3	3	7	7	1,886	0.0	finalized
4	127.77	80.8	3	3	11	11	6,877	0.0	finalized
5	133.46	84.6	3	3	7	7	2,221	0.0	finalized
6	123.10	77.9	3	3	9	9	3,693	0.0	finalized
7	124.73	78.9	3	3	6	6	1,957	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

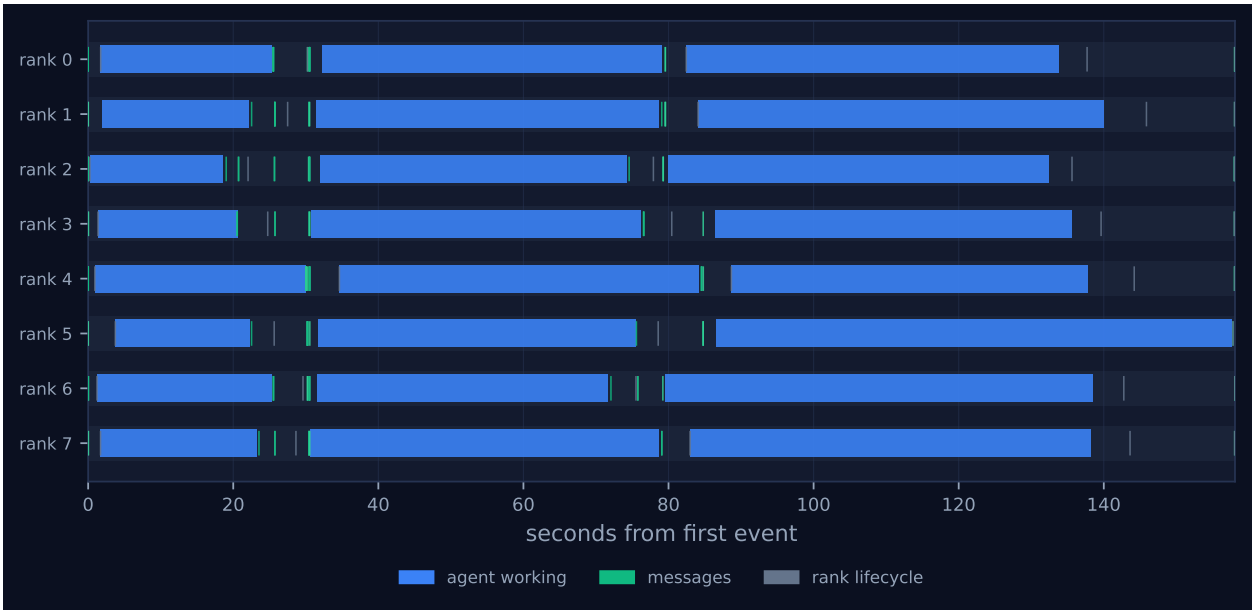

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

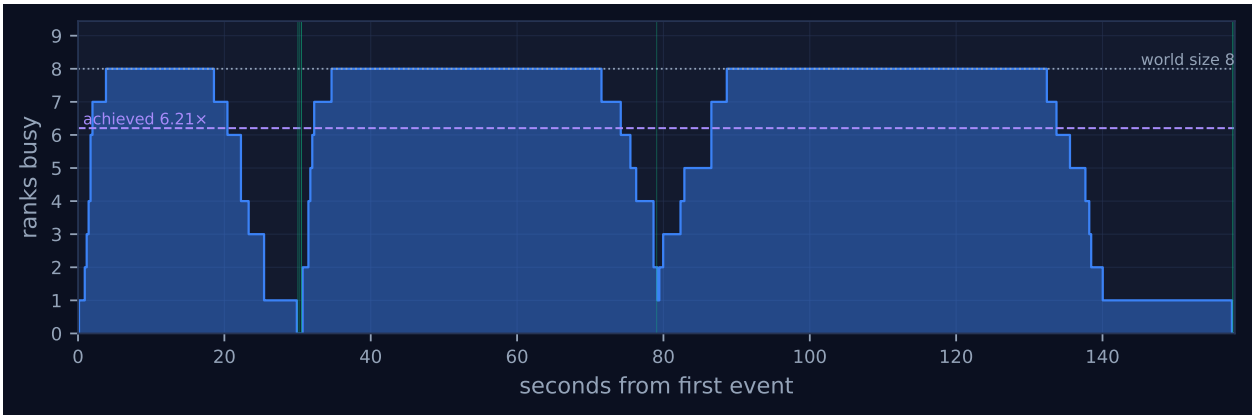

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

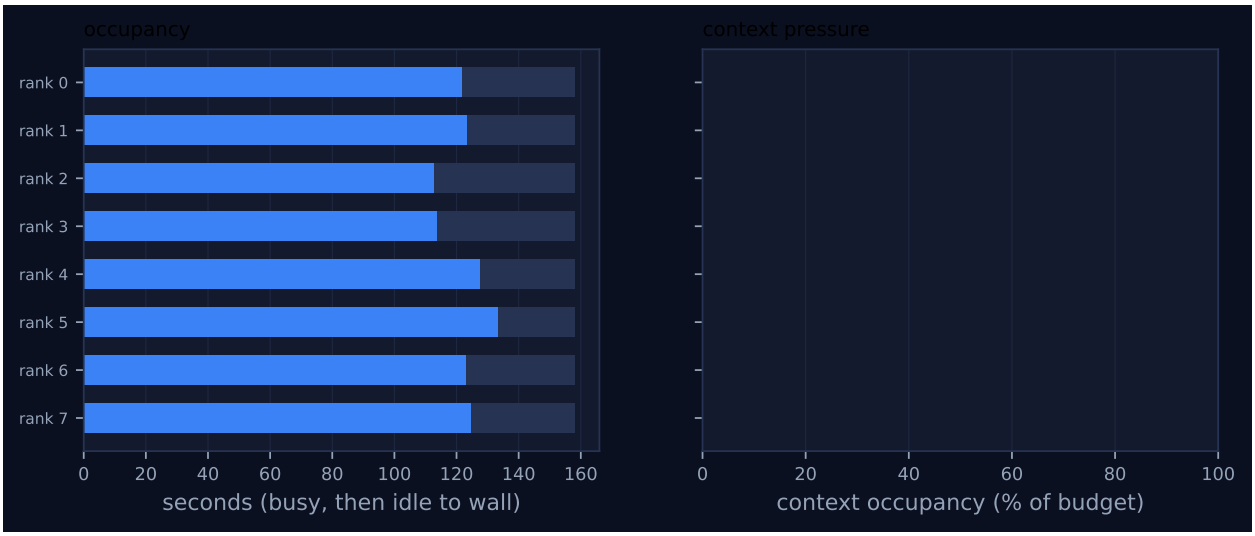

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	mod
scatter	binomial	decompose	8	8	3	3	7	7	11,594	0.03	0.03	✓
allreduce	recursive_doubling	glossary	8	8	3	3	24	24	12,353	11.36	0.42	✓
reduce	binomial	register	8	8	3	3	7	7	285	0.48	0.29	✓
bcast	binomial	register	8	8	3	3	7	7	1,274	0.32	0.03	✓
allreduce	reduce_bcast	register	8	8	6	6	14	14	0	0.49	0.03	✓
halo_exchange	?	seams	8	8	0	–	14	–	0	0.00	5.72	–
barrier	central	pre-gather	8	8	0	2	0	0	0	25.20	0.23	✓
gather	binomial	collect	8	8	3	3	7	7	5,079	0.13	0.27	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

The timeline in Figure 1 has exactly three blue bars per lane and nothing else of any width, and that is the whole shape of the run. The three bars are the three agent waves the harness prescribes — terminology extraction, translation, boundary revision — and the 24 `agent.call` events in the log are precisely three per rank. Everything the experiment is nominally about happens in the hairline gaps between the bars, where the green message ticks cluster. The collective under study is one of those hairlines. Read against the stats row, the proportion is stark: agent latency has a p50 of 48.52 s per invocation and the entire glossary allreduce contributed 0.430 s of critical path.

The first hairline is the glossary agreement, and its structure is worth unpacking because the figure

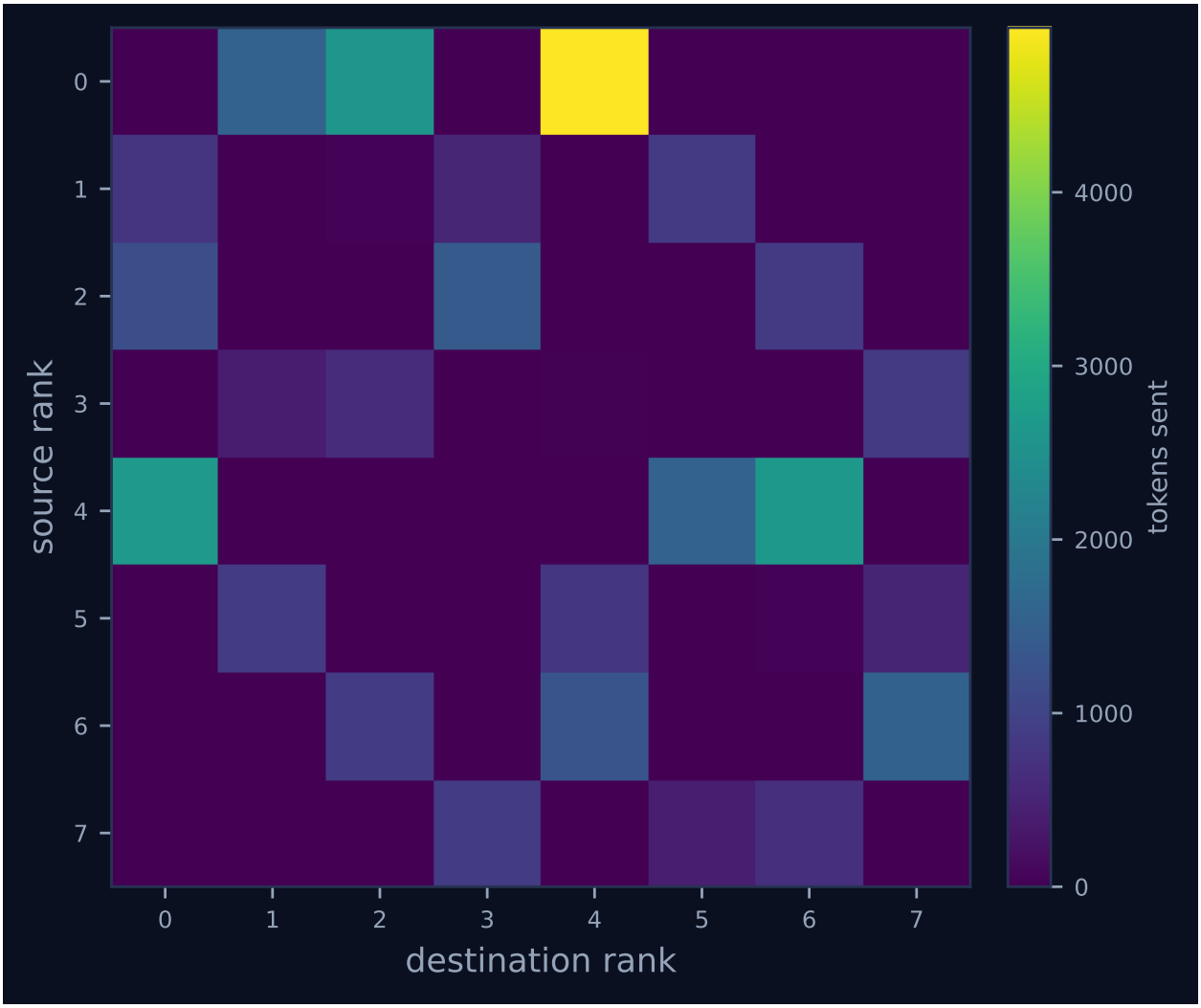

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

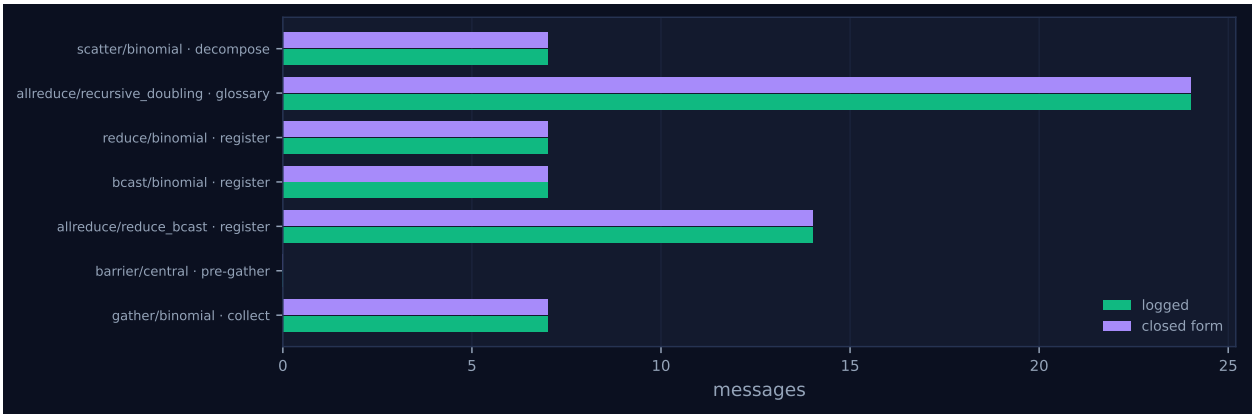

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

understates it. Each rank's `coll.allreduce` event carries the time it spent blocked inside the call, so subtracting that from the event timestamp recovers the moment it entered: ranks arrive at 19.03, 20.55, 22.53, 22.53, 23.54, 25.53, 25.54 and 30.03 s, an 11.00 s spread, and the last completion is at 30.46 s. The invocation therefore spans 11.43 s of which 11.00 s (96.2%) is one rank's terminology agent running long: rank 4 extracted 46 local terms, the most of any rank, took 30.01 s to do it, and consequently waited 0.00 s in the allreduce while rank 2, which arrived first, waited 11.36 s. In the timeline this is visible as the staggered right edges of the first wave — rank 4's opening bar is the one that extends furthest, and rank 2's is the shortest with the widest tick cluster after it. *The collective did not cost 11.36 s; the straggler did*, and the same reading applies to the baseline, whose glossary allreduce spans 10.49 s with a 10.00 s arrival spread. Isolating the algorithm means measuring from the last arrival to the last completion, which is 0.430 s here against 0.489 s for `reduce_bcast`.

The right-hand end of the figure is the run's real inefficiency and has nothing to do with the allreduce. Seven lanes finish their third bar between 132 and 146 s; rank 5's third bar runs to 157.81 s, visibly longer than any other bar in the picture, and its revision agent call took 73.02 s against a p50 of 48.52 s. The seven ranks that finished first sat in the pre-gather barrier for between 17.95 and 25.20 s waiting for it — the barrier's reported wall time is the straggler's queue, and rank 5's own entry records 0.0001 s because it arrived last. This single tail is most of the 22.4% idle fraction and all of the 14.8% serial fraction in the headline table, and it is why Figure 2 drops from eight busy ranks to one over the final 12 seconds while the dashed achieved-parallelism line settles at 6.21×.

Two figures are informative mostly by what they fail to show. Figure 4 is dominated by a single bright cell, $0 \to 4$ at 4,948 tokens, which is the rendezvous block of the `scatterv` decomposition and not a collective at all; the recursive doubling exchange appears only as faint symmetric off-diagonal cells of 361 to 847 tokens each. The algorithm's signature is nonetheless present if one looks at the *support* rather than the intensity: this run touches 15 distinct unordered rank pairs against the baseline's 10, and the five new ones — 1−3, 5−7, 1−5, 2−6, 3−7 — are exactly the hypercube edges at distances 2 and 4 that a binomial tree never uses. The right-hand panel of Figure 3 is flat at 0% for all eight ranks because every collective ran with `admit=False` and `tokens_admitted` is zero everywhere; this run exercises the transport, not the context accounting, and the panel should be read as silent rather than as reassuring.

7 Interpretation

What is true by construction

Three things follow from the configuration rather than from anything the run discovered. First, only the *glossary* allreduce changed algorithm: the harness threads `--allreduce-alg` into the glossary call alone, and the register agreement immediately afterwards takes the default. The run therefore carries both algorithms on the same fabric with the same ranks minutes apart, which is why the collectives panel of the viewer lists `recursive_doubling, reduce_bcast` under a single `allreduce` row with 16 invocations. That is a useful control on the mechanism but not on the cost, because the register agreement moves 1,649 logged tokens against the glossary's 12,709; the cost comparison has to be made against `real-tr-p8-full`, and it is.

Second, there was no divergence risk. `UNION` is declared `Associativity.EXACT`, and `Op.lossy` is defined as "associativity is not `EXACT`", so the `divergence_risk` flag that recursive doubling writes into the trace is `False` on all eight events. That flag is not a claim about this run's data; it is

a restatement of the operator's declared algebra. The empirical check is the harness's per-rank glossary digest, and it is unanimous: eight ranks, 206 terms, `c8b0a1300f57` at every one of them. The allreduce postcondition held exactly. Any statement that this run demonstrates recursive doubling to be divergence-safe would be overclaiming — it demonstrates that an exact operator is order-insensitive, which was already a theorem.

Third, $p = 8$ is a power of two, so the non-power-of-two remainder path was not entered. The trace confirms this directly rather than by inference: every message the allreduce sent is tagged `_ampi:allreduce:0:2:k` with $k \in \{0, 1, 2\}$, eight messages per round, and there is no `:pre` or `:post` tag anywhere in the log. This matters because that path carried a real accounting bug — both partners exchange via `sendrecv` but only the odd-numbered rank recorded its send — and the fix is visible in the current source as an explicit comment on the even branch's `tr.sent` call. Summing self-reports against tagged sends across the microbenchmark family reproduces the bug's exact footprint: the shortfall is 0 at $p \in \{2, 4, 8, 16, 32\}$ and precisely $rem = p - 2^{\lfloor \log_2 p \rfloor}$ at every other size (+1 at $p = 3, 5$; +2 at $6, 10$; +3 at 7; +4 at $12, 20$; +8 at 24) — that is, in the eight `mb-free` microbenchmark runs of `coll-allreduce-recursive_doubling` at those p. In every one of those cases the *logged* count equals the closed form, so the cost model in `cost.py` — whose message term is $3\,rem + 2^{\lfloor \log_2 p \rfloor} \lfloor \log_2 p \rfloor$, counting both halves of the pre-exchange — was right and the implementation's hand-maintained counter was wrong. This run contributes nothing to that question and should not be cited on it.

What the run measures

The substance is the measured cost triple against the `reduce_bcast` baseline, and it does not line up the way the algorithm's reputation suggests.

Rounds and latency. Three rounds against six, as predicted, and the closed form is confirmed by the tags: three distinct round suffixes with eight sends each. The saving is 59 ms, or 0.04% of the 158.08 s wall. Recursive doubling is latency-optimal in a model where α is the per-hop cost, and `algorithms.py` argues at length that for agent ranks α is an agent invocation of tens of seconds, which is what makes logarithmic depth worth more here than in MPI. The argument is sound and does not apply to this run, because `UNION` executes as local host code: the operator is free, so the rounds it saves are fabric hops of roughly 20 ms and the whole latency case evaporates. The condition under which recursive doubling's round advantage is worth having is precisely the condition under which its results can diverge.

Messages and volume. 24 messages against 14, a 71% increase, and the run total rises from 56 to 66. Tokens go the other way: 12,709 against the baseline's 13,765 (reduce 2,642 plus broadcast 11,123), a 7.7% *reduction*, and the run total is essentially unchanged at 31,373 against 31,603. The 12,709 figure is not in `metrics.json` — for the reason given below, this invocation has no attributed traffic — so I summed the `tokens` field of every `msg.send` event whose tag begins `_ampi:allreduce:0:2`; it exceeds the implementation's own 12,353 by 356 because the self-report counts only the accumulator while the wire carries the depth and weight fields alongside it. The mechanism is visible per round — mean message size 263.9, 464.8 and 860.0 tokens as the partial unions accumulate — and the point is that recursive doubling never sends the complete glossary at all, whereas the binomial broadcast ships the whole 1,589-token union down each of seven edges. Note that this contradicts the closed-form volume term, which assumes a payload of fixed size n and therefore predicts $24n$ against $14n$: the union's accumulator grows sublinearly along the tree (factors of 1.76 and 1.85 rather than 2, because duplicate terms merge), and for a set-like operator the model's volume

ranking is simply the wrong shape. The message-count check, which is what the analysis actually validates, is unaffected.

Where the traffic sits. This is the clearest win and it is a distributional one. Summing rank 0's outbound `msg.send` events under the glossary tags in each trace, its egress falls from 4,767 tokens to 1,743, a 63.4% reduction, in the same three messages — so the improvement is in volume, not in fan-out, because `reduce_bcast` already used a *binomial* broadcast and the root's degree was $\log_2 p$ either way. Across the whole run the per-rank token spread narrows from 20.96× between the busiest and quietest sender to 4.83× (coefficient of variation 1.112 to 0.681), and the message spread from 2.75× to 1.83×. The load imbalance in busy time also falls, 1.146 to 1.088, though that is agent latency and not attributable to the algorithm. Recursive doubling's "no root bottleneck" claim is therefore real but narrower than advertised at $p = 8$: it removes an asymmetry in bytes, not one in message count, and only because the baseline's root was already logarithmic.

Fold depth, which is the interesting one. All eight ranks report `fold_depth: 3` for the recursive doubling allreduce, which is true by construction — every rank performs one fold per round. The comparison quantity is the depth at whichever rank holds the result, and for `reduce_bcast` that is the root: rank 0's `coll.reduce` event reports `fold_depth: 3` in every one of the three translation runs, with the rest of the tree at $0, 1, 0, 2, 0, 1, 0$ over the binomial levels. The maxima are equal, and they must be: `cost.py` gives both algorithms a depth term of $\lceil \log_2 p \rceil$. So the extra three rounds `reduce_bcast` spends are pure broadcast, which performs no operator applications and — because AgentMPI forwards content-addressed handles rather than regenerated text — is drift-free at any tree depth. The consequence for a lossy operator is the opposite of the intuition that "fewer rounds means less compounding": recursive doubling composes to the same depth, so it improves fidelity not at all, while raising the total application count from $p - 1 = 7$ to $p \log_2 p = 24$. The semantic sibling `real-tr-p8-semanticgloss` prices that: it made 31 agent calls against this run's 24, the 7 extra being exactly its $p - 1$ semantic folds, and it cost $0.3536 against $0.2599, so a fold ran about $0.013. A semantic recursive doubling would need 24 of them for the same depth-3 result, with each rank on its own critical path of three sequential model calls. Against a run whose glossary reduce already consumed 980.7 s of a 1,152.9 s wall, that is a large bill for a 59 ms saving and a weakened postcondition.

One caution about that comparison, which is a second fixed bug visible in a sibling trace rather than in mine. The baseline's `coll.allreduce` events do not report the reduce's fold depth at all: their per-rank values are $0, 1, 1, 2, 1, 2, 2, 3$, which is exactly the per-rank `tree_depth` of the following broadcast. In `real-tr-p8-full` the broadcast writes its tree depth into `fold_depth`, so it overwrites the reduce's figure before the outer allreduce reads it; taken literally, that trace says the root performed zero lossy compositions. The current source fixes this at both ends — the binomial broadcast now sets `fold_depth = 0` and records the distance from the root under `tree_depth`, with a comment explaining that a broadcast is drift-free at any depth because it forwards handles, and `allreduce` reads `LAST_STATS` before the broadcast can clobber it. My run and `real-tr-p8-semanticgloss` were recorded after the fix and report $3, 0, 1, 0, 2, 0, 1, 0$ correctly for their `reduce_bcast` allreduces, so the three siblings are not all the same vintage. The aggregate in `metrics.json` survives only by coincidence: it takes the maximum over ranks, and the deepest broadcast tree path happens to equal $\lceil \log_2 p \rceil$ as well. Any per-rank fidelity claim drawn from a pre-fix `reduce_bcast` trace is measuring the wrong tree.

The divergence question, and the hole in the sweep

Recursive doubling has every rank fold in its own order, and `allreduce` narrows that with one rule: when the operator is not commutative the accumulator is assembled with the lower-ranked operand on the left, `op.fn(acc, body["acc"])` if `comm.rank < partner` and the two swapped otherwise. Tracing that through the hypercube shows every rank evaluating the identical parenthesisation,

$$\big((a_0{\cdot}a_1){\cdot}(a_2{\cdot}a_3)\big) \;\cdot\; \big((a_4{\cdot}a_5){\cdot}(a_6{\cdot}a_7)\big),$$

in rank order at every level. A *deterministic* lossy operator therefore cannot diverge under this implementation — the ordering rule removes that failure mode structurally, which is a real and underadvertised design decision. What survives is nondeterminism: eight ranks evaluating the same expression with eight independent model invocations get eight different answers, and no ordering rule can prevent it. Note that the trace flag keys on `op.lossy`, i.e. on declared associativity, not on determinism; it is conservative in the right direction, over-flagging a deterministic lossy operator rather than missing a nondeterministic one.

Placed in the ablation ladder, this run and its siblings bracket the hazard without touching it. `real-tr-p8-semanticgloss` varies the operator alone and its eight ranks also agreed unanimously (`c908d2c6e40f`, 203 terms), because `reduce_bcast` computes one canonical result and broadcasts it by handle. This run varies the algorithm alone and its eight ranks agreed unanimously, because `UNION` is exact. Neither factor causes divergence on its own; only their conjunction can, and `semantic` × `recursive_doubling` is the one cell of that 2×2 the campaign does not contain. The quality metrics are correspondingly uninformative for the comparison — coverage 1.0, consistency 1.0, zero divergent entities, rendering verification 1.0 and paragraph fidelity 1.0 in both this run and the baseline — and should be read as confirming that the harness worked, not as evidence about the algorithm.

Triaging the flagged findings

The 31 reattachments are by construction and correctly marked `[ok]`. The log holds 39 `rank.init` events, an initial `broker` incarnation plus a fresh `worker` for each phase boundary, reaching incarnation 5 on seven ranks and 4 on rank 5; the rank roles, mailboxes and context accounts persisted across all of them, which is what the eight unbroken lanes in Figure 1 show.

The `halo_exchange` warning — 0 messages reported against 14 logged — is expected for this trace and is not a live defect. It is a vintage artefact: the recorded event carries only `label`, `left` and `right`, whereas the current `topology.py` emits `algorithm="sendrecv"`, `rounds` and `messages_sent=sent`. The 14 logged sends are exactly what the fixed version would report (two per interior rank, one at each end of the open 1-D Cartesian line, $2 \times 6 + 2 = 14$), so the warning would disappear on a re-recording. The identical warning appears in `real-tr-p8-full` and `real-tr-p8-semanticgloss`, confirming it is independent of the change under test. The row would still show – in the cost-model column, because no closed form for `halo_exchange` exists.

A defect in the analysis tooling, not in the protocol

The claim that "all 7 checkable collectives sent exactly the number of messages their closed-form cost expression predicts" is true, but for the collective this run exists to study it is weaker than the document represents. The table's footnote and the "logged" legend in Figure 5 both assert that the

compared quantity is the traffic the fabric recorded, attributed by internal tag. For both `allreduce` rows, `logged_messages` is `null` in `metrics.json`, and the table, the figure and the model check all silently fall back to the collective's own self-report — so the check that is documented as independent is, here, a restatement.

The cause is precise. `_attribute_logged_traffic` groups tagged sends by (op, generation, epoch) and then refuses to match when the number of groups for an op differs from the number of invocations of that op, to avoid pairing the wrong records. This run has two `allreduce` invocations but only one tagged epoch, because `reduce_bcast` takes no epoch of its own and its traffic is tagged under the nested `reduce` and `bcast`. So $1 \neq 2$, the function bails, and both rows lose attribution: `accounting_agrees` becomes `null`, which also exempts them from the misreporting check entirely. This is the same guard whose docstring names "recursive doubling at non-power-of-two sizes" as the miscount it exists to catch, and it is disabled by the mere presence of a second, composed allreduce in the same run. In the microbenchmarks it works: `mb-free__coll-allreduce-recursive_doubling-5` has exactly one allreduce and duly reports `accounting_agrees: false` with 10 self-reported against 11 logged. The fix is small and the ingredients are already present — `composed_of` is populated by `_attribute_nested_traffic`, which runs first, so excluding composed invocations before comparing counts (or matching on epoch instead of on time-ordered position) restores attribution. I verified this run's allreduce by hand against the log: 24 tagged sends under `_ampi:allreduce:0:2`, eight in each of three rounds, equal to the self-report and to the closed form. The conclusion in the table is right; the tool did not establish it.

A second measurement artefact bore directly on this comparison and has since been corrected, which is worth recording because it inverted the answer. The coordination figure used to be the sum of every invocation's longest per-rank wait divided by wall time, summing composed invocations together with the constituents they delegate to — so a `reduce_bcast` allreduce was charged three times, once as itself and once for each of the nested reduce and broadcast. That sum is still recoverable from the per-invocation wall times in `metrics.json`: 38.020 s here against 158.08 s of wall, and 59.475 s against 166.78 s in the baseline, which is 24.1% against 35.7% and reads as recursive doubling cutting coordination overhead by a third. It was not a proportion at all — on the semantic sibling the same arithmetic gives 2,982.8 s against a 1,152.9 s wall, a "share" of 258.7% — and the double counting fell specifically on `reduce_bcast`, which is to say specifically on the arm this run is compared against. The current artefacts report the corrected pair of measures, and the corrected pair says the opposite: coordination share, now rank-seconds blocked over rank-seconds available, is 16.1% here against 16.4% in the baseline, and coordination in flight, the union of the intervals during which any rank was inside a collective, is 23.3% against 26.9%. The residual 3.6 points is almost entirely the pre-gather barrier (25.20 s against 33.71 s), which is straggler variance. On the corrected metric the algorithm change is invisible in coordination overhead, as it is in every other aggregate.

8 Threats to this reading

The effect is far smaller than the noise it sits in. The run is 158.08 s against the baseline's 166.78 s, and it would be wrong to credit 8.7 s to the algorithm when its measured contribution to the critical path is 59 ms. The two runs' agent latencies differ substantially at random — per-rank p50 spans 41.5–53.0 s here against 36.5–66.0 s in the baseline — and the glossary allreduce's own wall time was *longer* in this run (11.36 s against 10.24 s) purely because its first arriver came 0.5 s

earlier and its last 0.5 s later. Achieved parallelism (6.205 against 6.195), parallel efficiency (77.6% against 77.4%) and idle fraction (22.4% against 22.6%) are indistinguishable. Every aggregate in the headline table is dominated by agent latency, and a single pair of runs cannot separate a 59 ms effect from it.

The digest agreement is a weaker test than it appears. It is tempting to read eight identical digests as evidence that the canonical ordering rule works, but this run never exercised the order-sensitive path. `UNION`'s conflict branch already sorts merged proposals deterministically before returning them, and the harness resolves any surviving list with `sorted(v)[0]` — two independent layers of order-independence on top of an operator that is exact, commutative and idempotent. Worse, `n_conflicting_terms` is 0 on all eight ranks in this run, against 4 in the baseline, so no term ever had competing renderings to resolve. The digest was computed on a value that could not have differed. A real test of the ordering rule needs a deterministic *non-commutative* operator such as `CONCAT`, which this run does not contain.

The token comparison measures notional volume, not cost to an agent. `tokens_admitted` and `context_used` are zero on every rank, so the 7.7% volume reduction is wire traffic that no model ever read, and the right-hand panel of Figure 3 is uninformative rather than healthy. The reduction also depends on the glossary union growing sublinearly, which is a property of this text's terminology and not of the algorithm; for a fixed-size payload the closed forms are correct and recursive doubling would send 71% more volume, not 7.7% less. Do not generalise the volume result past set-like operators.

The fold-depth numbers are self-reported. Both algorithms compute `fold_depth` inside the implementation and write it to the trace; nothing in the analysis derives it independently from the message pattern, and for recursive doubling the uniform 3 across all ranks is true by construction rather than observed. The equality of the two maxima is corroborated by `cost.py`'s depth terms, which is agreement between two parts of the same codebase, not a measurement.

This is a power of two, and the campaign's agent populations mostly are not. The remainder path is where the algorithm's complexity and its confirmed under-report of *rem* messages lived, and $p = 8$ cannot reach it. Anything this run says about recursive doubling's accounting applies only to the clean case.

Two cross-run comparisons need care. The baseline's log contains ten ranks for a world size of eight — ranks 8 and 14 appear with zero busy time — and I excluded those two from the dispersion statistics; they do not affect its headline aggregates, which filter zero-busy ranks, but any per-rank comparison against `real-tr-p8-full` has to exclude them explicitly. And the $0.013-per-semantic-fold figure used to price a hypothetical semantic recursive doubling comes from differencing two runs whose translation phases also differed, so it is an order-of-magnitude estimate rather than a measurement.

Finally, the executors were real. `metrics.json` records eight `broker` and 31 `worker` executors with agent latencies in the tens of seconds, so this run does report on agent behaviour rather than on a surrogate. That cuts the other way for reproducibility: the term sheets, and hence the glossary digest, are not reproducible across runs — the baseline agreed on a different 206-term glossary (`0233d780cdb6`) from the same source text. Only the *agreement* is a protocol property; its content is not.